

SENA — ADSO 2026

Clase 33

# Diseño Técnico del Proyecto

---

API de SendGrid — Notificaciones por Email

*Texticode — Sistema de Gestión Textil*

## **Aprendices**

Kevin Stevan Wagner  
Kevin Andres Castro  
Andres Felipe Lancheros  
Camilo Andres Tibambre

## **Instructor**

Ing. Edwin Marín

## **Fecha**

Mayo 2026

## **Programa**

ADSO 2026

# 1. Arquitectura 3 Capas

SendGrid se integra como servicio externo de email transaccional. El flujo pasa por 3 capas: el frontend dispara la acción, el backend valida y llama a emailService.js, y este se comunica con la API de SendGrid via HTTPS.

|                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|
| <b>FRONTEND</b><br>Vue.js 3 · LoginView.vue, GestionProduccion.vue, ReportarAvances.vue · useNotificaciones.js    |
| ↓ HTTP / REST (x-api-key)                                                                                         |
| <b>BACKEND</b><br>Node.js + Express · routes/notificaciones.js · auth.route.js · services/emailService.js         |
| ↓ HTTPS (Authorization: Bearer SG.xxxxx)                                                                          |
| <b>SENDGRID API</b><br>api.sendgrid.com · POST /v3/mail/send · Plan gratuito 100 emails/día · Entrega < 1 segundo |

## FLUJO DE DATOS — RECUPERACIÓN DE CONTRASEÑA

| Capa            | Acción                                                                                          |
|-----------------|-------------------------------------------------------------------------------------------------|
| Frontend        | LoginView.vue → POST /api/auth/recuperar-contrasena { email: 'usuario@gmail.com' }              |
| Backend Auth    | auth.route.js valida formato email con regex, busca usuario en BD                               |
| Backend Service | Genera token seguro con crypto.randomBytes(32), guarda en tokens_recuperacion con expiración 1h |
| SendGrid        | emailService.js llama sgMail.send() → POST https://api.sendgrid.com/v3/mail/send                |
| Response        | 200 OK + { mensaje: 'Si el correo está registrado, recibirás el enlace.' }                      |

## 2. Contrato Técnico API

Los 4 contratos de la integración SendGrid en Texticode, documentados con evidencia real de pruebas en Thunder Client.

### 2.1 POST /api/auth/recuperar-contrasena

**POST /api/auth/recuperar-contrasena**

*Genera un token seguro y envía email de recuperación de contraseña via SendGrid. Expira en 1 hora.*

#### REQUEST

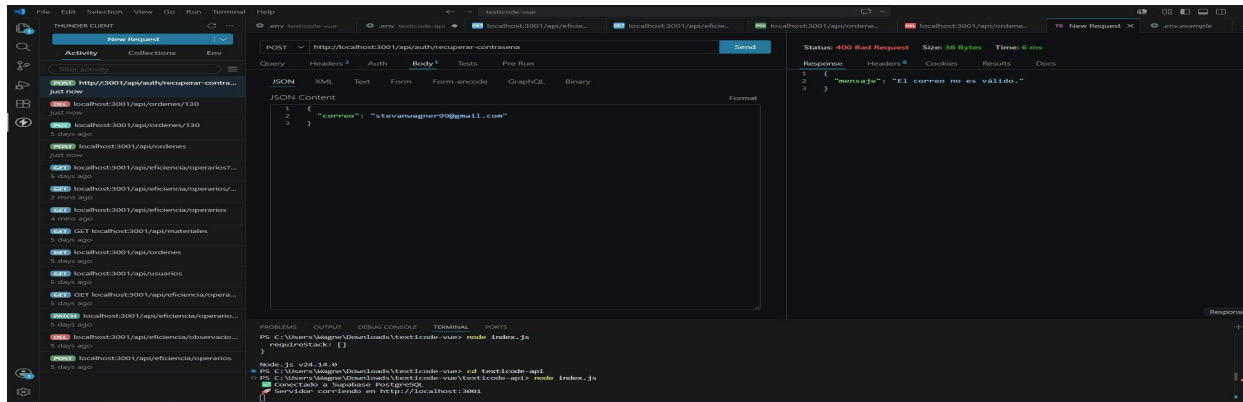
```
POST /api/auth/recuperar-contrasena HTTP/1.1
Host: localhost:3001
Content-Type: application/json

{
  "email": "stevanwagner1@gmail.com"
}

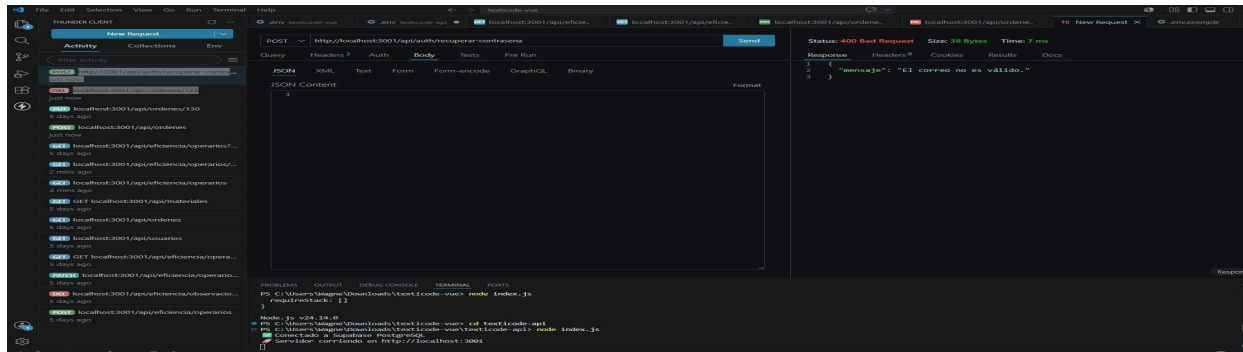
// Campo requerido: email (formato válido correo@dominio.com)
// Respuesta siempre 200 por seguridad (no revela si el email existe)
```

#### EVIDENCIA THUNDER CLIENT

##### Prueba 1 — Email registrado en BD → 200 OK



##### Prueba 2 — Body vacío (email inválido) → 400 Bad Request



## 2.2 POST /api/notificaciones/estado

### POST /api/notificaciones/estado

Notifica al cliente por email cuando el estado de su orden cambia. Llamado automáticamente desde el backend.

#### REQUEST

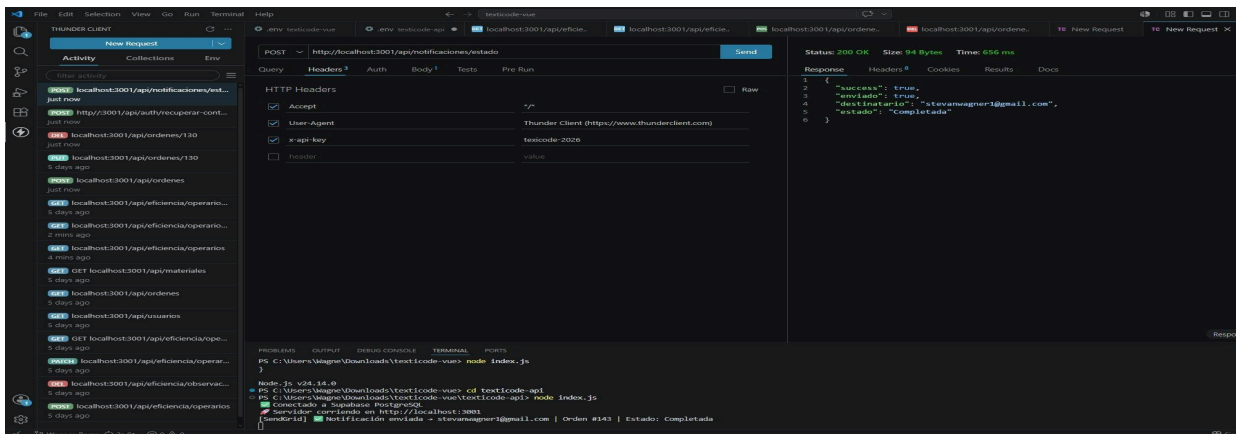
```
POST /api/notificaciones/estado HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "ordenId": 143,
  "clienteEmail": "stevanwagner1@gmail.com",
  "clienteNombre": "Stevan Wagner",
  "estado": "Completada",
  "productos": ["Boxer", "Camisas"]
}

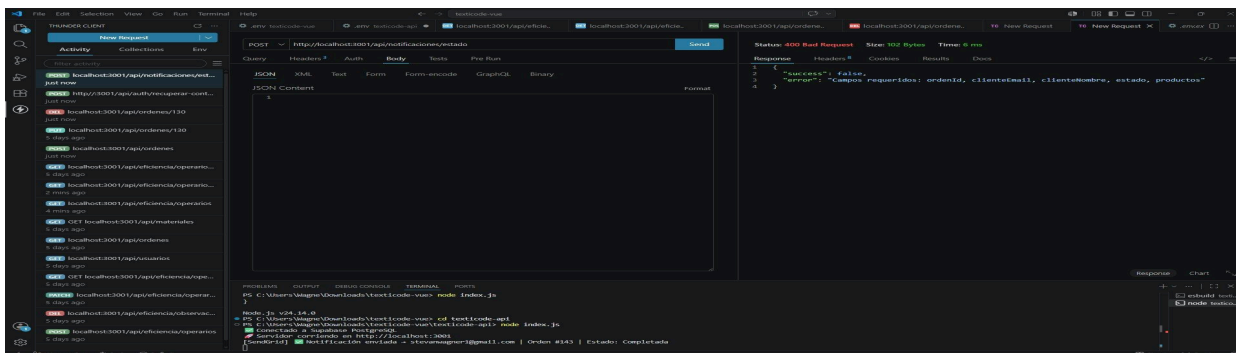
// Campos requeridos: ordenId, clienteEmail, clienteNombre, estado, productos
// observaciones es opcional
```

#### EVIDENCIA THUNDER CLIENT

##### Prueba 1 — Body completo y correcto → 200 OK



##### Prueba 2 — Body vacío (campos faltantes) → 400 Bad Request



## 2.3 POST /api/notificaciones/tarea

### POST /api/notificaciones/tarea

*Notifica al operario por email cuando se le asigna una nueva tarea de producción.*

## REQUEST

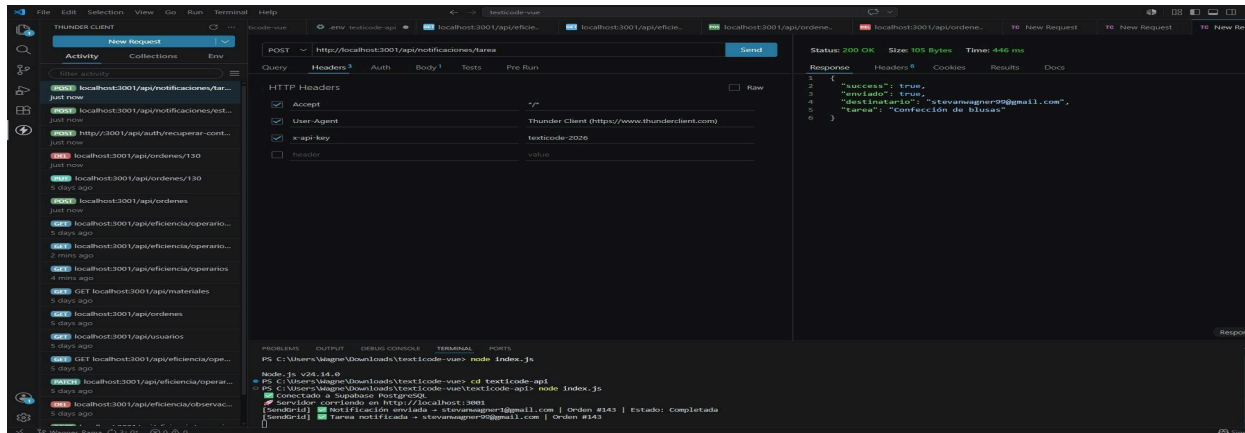
```
POST /api/notificaciones/tarea HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "operarioEmail": "stevanwagner99@gmail.com",
  "operarioNombre": "Stevan Wagner",
  "tarea": "Confección de blusas",
  "ordenId": 143,
  "prioridad": "Alta",
  "fechaLimite": "2026-06-30"
}

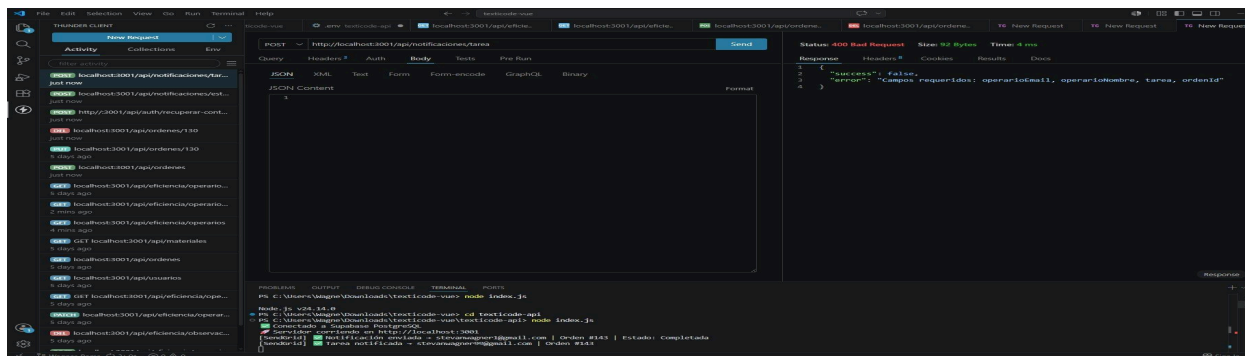
// Campos requeridos: operarioEmail, operarioNombre, tarea, ordenId
// prioridad y fechaLimite son opcionales
```

## EVIDENCIA THUNDER CLIENT

### Prueba 1 — Body completo y correcto → 200 OK



### Prueba 2 — Body vacío (campos faltantes) → 400 Bad Request



## 2.4 GET /api/notificaciones/estadisticas

### GET /api/notificaciones/estadisticas

Consulta las estadísticas de emails enviados desde la cuenta SendGrid. Soporta filtro por fecha.

#### REQUEST

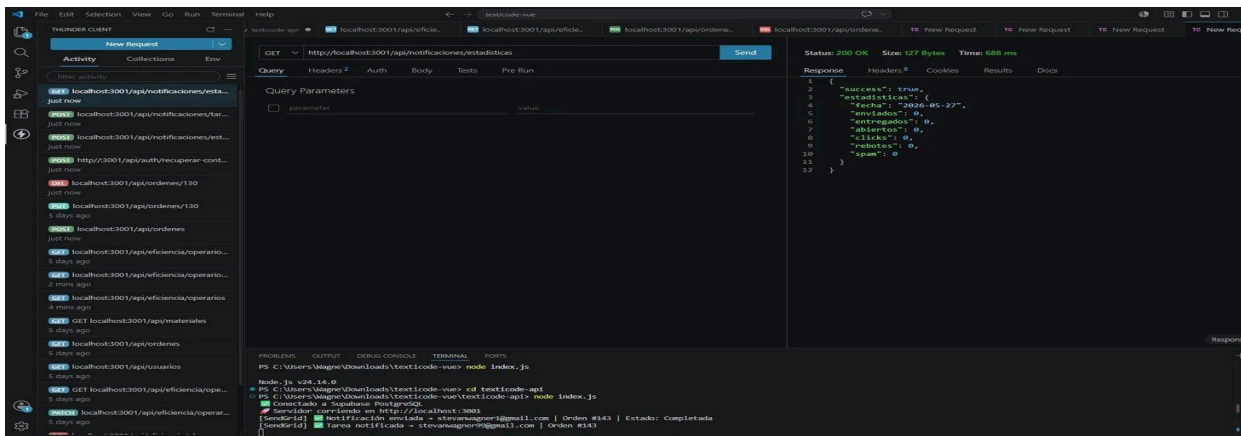
```
GET /api/notificaciones/estadisticas HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

// Query param opcional:
?fecha=2026-05-26 → filtra estadísticas por fecha (formato YYYY-MM-DD)

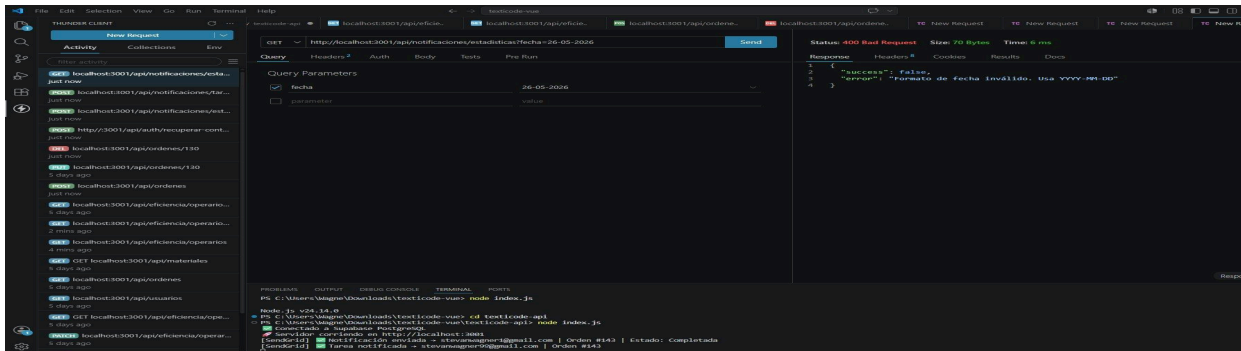
// Respuesta incluye: emails enviados, abiertos, clicks, rebotes
// Datos obtenidos directamente de GET https://api.sendgrid.com/v3/stats
```

#### EVIDENCIA THUNDER CLIENT

##### Prueba 1 — Sin query param (estadísticas del día) → 200 OK

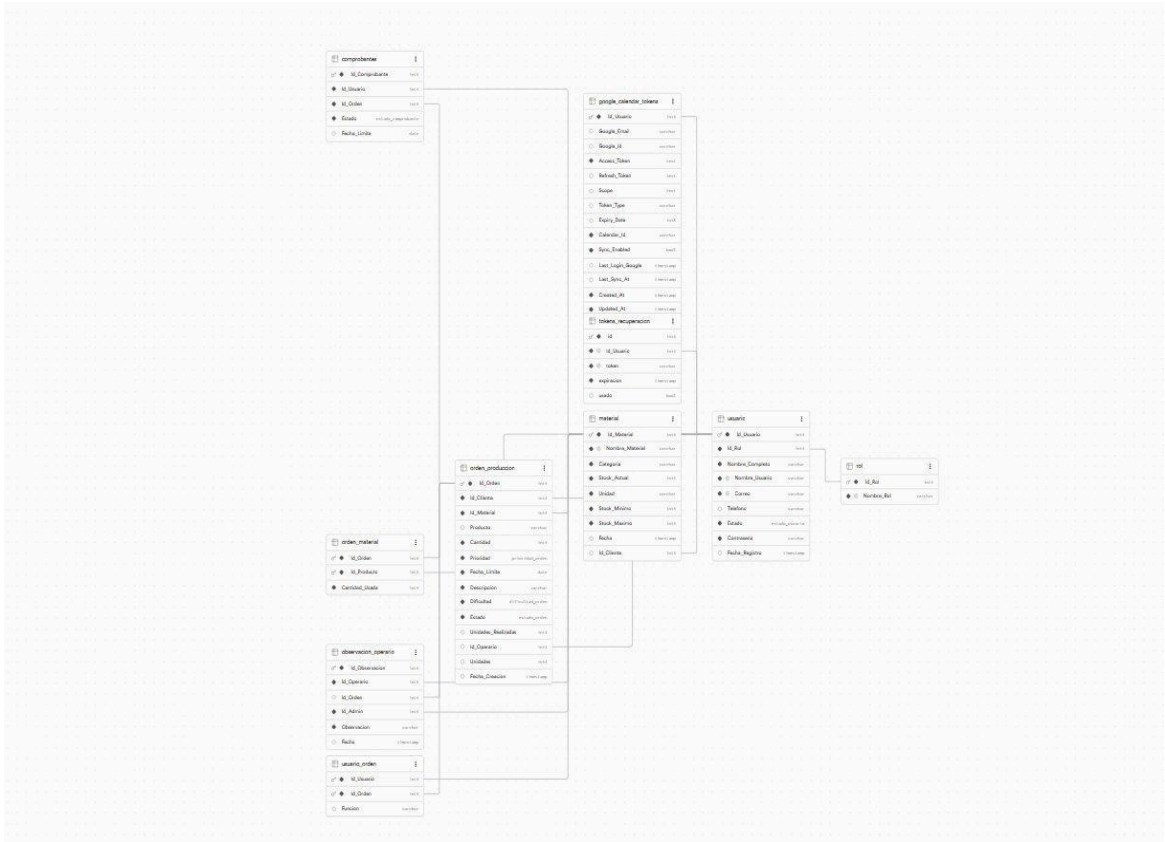


##### Prueba 2 — Fecha con formato incorrecto (?fecha=26-05-2026) → 400 Bad Request



### 3. Diagrama Entidad-Relación

La integración SendGrid en Texticode involucra 4 tablas de BD más la API externa. La tabla tokens\_recuperacion es la única creada exclusivamente para esta integración.



| Tabla / Entidad        | Campos relevantes                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------|
| usuario                | Id_Usuario (PK) · Nombre_Completo · Correo · Id_Rol (FK) · Estado                                 |
| rol                    | Id_Rol (PK) · Nombre_Rol (admin   operario   cliente)                                             |
| orden_produccion       | Id_Orden (PK) · Id_Cliente (FK) · Id_Operario (FK) · Producto · Estado · Prioridad · Fecha_Limite |
| tokens_recuperacion    | id (PK) · Id_Usuario (FK) · token (UNIQUE) · expiracion · usado                                   |
| SendGrid /v3/mail/send | to · from · subject · html (template email) — API externa, no tabla BD                            |
| emailService.js        | notificarCambioEstado() · notificarTareaOperario() · enviarComprobante() · obtenerEstadisticas()  |

#### RELACIONES CLAVE

| Origen  | Relación | Destino             | Descripción                                      |
|---------|----------|---------------------|--------------------------------------------------|
| usuario | 1 → 1    | tokens_recuperacion | Un usuario tiene un token de recuperación activo |
| usuario | 1 → N    | orden_produccion    | Un cliente tiene múltiples órdenes               |

| Origen              | Relación  | Destino        | Descripción                                    |
|---------------------|-----------|----------------|------------------------------------------------|
| orden_produccion    | dispara → | SendGrid email | Cambio de estado → notificación al cliente     |
| usuario (operario)  | dispara → | SendGrid email | Asignación de tarea → notificación al operario |
| tokens_recuperacion | dispara → | SendGrid email | Token generado → enlace de reseteo al usuario  |
| rol                 | 1 → N     | usuario        | Un rol agrupa múltiples usuarios               |



## 4. Validaciones y Manejo de Errores

Todos los endpoints de notificaciones validan campos obligatorios antes de llamar a SendGrid, evitando costos innecesarios de API y errores en producción.

| Escenario                    | Validación en código                                            | HTTP             |
|------------------------------|-----------------------------------------------------------------|------------------|
| API Key ausente              | <code>!req.headers['x-api-key']</code>                          | 401 Unauthorized |
| API Key incorrecta           | <code>key !== process.env.API_KEY_EFICIENCIA</code>             | 403 Forbidden    |
| Email inválido               | <code>!/^[\s@]+@[\s@]+\.[\s@]+\$/\.test(email) === false</code> | 400 Bad Request  |
| Campos requeridos faltantes  | <code>!ordenId    !clienteEmail    !clienteNombre...</code>     | 400 Bad Request  |
| Fecha con formato incorrecto | <code>!/^d{4}-d{2}-d{2}\$/\.test(fecha)</code>                  | 400 Bad Request  |
| Error de SendGrid API        | <code>error.response?.body?.errors[0].message</code>            | 500 Server Error |
| Error interno del servidor   | <code>catch(err) → log interno</code>                           | 500 Server Error |

### CONFIGURACIÓN SENDGRID (EMAILSERVICE.JS)

```
// services/emailService.js
import sgMail from '@sendgrid/mail'

// API Key desde .env - nunca hardcodeada en el código
sgMail.setApiKey(process.env.SENDGRID_API_KEY)

// Variables de entorno requeridas:
// SENDGRID_API_KEY=SG.qdwB9Ag5S76XC8rgjtqr1g...
// SENDGRID_FROM_EMAIL=stevanwagner1@gmail.com
// FRONTEND_URL=http://localhost:5173
```